ZAR HORIZON AI @WORK repository instructions

Purpose

This repository is a design-first, documentation-driven monorepo for ZAR HORIZON AI @WORK. The approved product and architecture decisions in `docs/products`, `docs/architecture`, `docs/database`, `docs/api`, and `docs/ux` are the source of truth. Do not treat the scaffold as a production-ready implementation.

Required reading before changes

Before modifying a subsystem, read the relevant design artifacts first:

- Shared platform and product boundaries: `docs/products/`
- Product and market decisions: `docs/market-research/`, `docs/customer-validation/`, `docs/use-cases/`, `docs/mvp/`
- Architecture and enforcement model: `docs/architecture/`
- Data and tenant isolation model: `docs/database/`
- API and tool contracts: `docs/api/`
- UX and permissions: `docs/ux/`
- Development standards and repo setup: `docs/development/`
- Repo baseline and local workflows: `README.md`

If a request conflicts with these documents, stop and ask for clarification instead of choosing an arbitrary implementation.

Repository-wide rules

- Follow the approved architecture, security boundaries, and product separation.
- Keep changes small, understandable, and testable.
- Prefer existing patterns and tooling over new dependencies or broader refactors.
- Do not introduce microservices, Kubernetes, or distributed infrastructure without explicit approval.
- Do not invent database entities, API contracts, agent tools, integrations, or business requirements.
- Do not silently change founder-approved decisions or security assumptions.
- Explain assumptions, trade-offs, and material risks before making a change that affects architecture or security.
- Keep documentation aligned with implementation changes that materially affect the repo.
- Never claim a build, test, migration, deployment, or integration succeeded without actually running and verifying it.
- Never expose credentials, API keys, tokens, customer data, or secrets in prompts, logs, generated files, or commit output.

Product separation

The repository intentionally separates the product domains:

- AI Business Partner: business operations, monetisation, planning, and execution workflows with deterministic financial logic and approval boundaries.
- AI Hacker: authorised security-assessment workflows that require explicit scope, evidence capture, and controlled execution.
- Shared platform: identity, workspace, organisation, tooling governance, approvals, audit, and shared infrastructure. Shared infrastructure does not mean shared authority.

Business Partner guardrails

- Must stay inside its own product permissions and data domain.
- Must not inherit offensive security tools or Hacker execution privileges.
- Financial calculations must be deterministic and auditable.
- External actions require approval policy enforcement.
- Recommendations must distinguish verified information, estimates, and assumptions.

AI Hacker guardrails

- Must stay inside its own security permissions and data domain.
- Requires explicit authorisation and a well-defined testing scope.
- Must not have unrestricted network, shell, or exploitation access.
- Must use controlled execution environments and capture evidence.
- High-risk actions require explicit approval.
- Must not inherit Business Partner financial or commercial permissions.

Change control and Git safety

- Inspect `git status` before substantial work.
- Never overwrite or delete unrelated user work.
- Avoid destructive Git commands.
- Do not commit, push, merge, or deploy without explicit authorisation.
- Never weaken security controls to satisfy a test or a convenience goal.
- Never modify production configuration without explicit approval.
- Avoid broad refactors when a focused change is enough.

Testing and verification policy

- Run the smallest relevant checks for the change.
- For frontend work, validate formatting, linting, type checking, and tests where available.
- For backend work, validate Python formatting, linting, type checking, and tests where available.
- For database or tenant-binding work, validate migration and isolation assumptions with the relevant checks.
- For security execution or approval logic, verify the permission and evidence model rather than assuming it is correct.
- If a check cannot run, say so clearly and do not substitute an assumed success result.

Scope

This repo is not a blank-slate app. It is a governed foundation for future implementation. Do not implement product features, create user-facing pages or components, add AI provider integrations, or modify the architecture or approved specs without explicit approval.